UOVの一般化と高機能化について

鈴木 俊博[1,2,a)]　伊藤 琢真[2]　内山 成憲[1]

概要：署名方式 UOV とその変種は耐量子計算機暗号の有力な候補である．本研究では, UOV のトラップドアを一般化することで, 一つの公開鍵に複数の秘密鍵が対応するような UOV の変種を提案し, その安全性と実用性について考察する．

キーワード：耐量子計算機暗号, 多変数公開鍵暗号, UOV, 高機能署名, 秘密分散

On a Generalization and Functional Enhancement of UOV

Toshihiro Suzuki[1,2,a)]　Takuma Ito[2]　Shigenori Uchiyama[1]

Abstract: The UOV signature scheme and its variants are promising candidates for post-quantum cryptography. In this work, by generalizing the trapdoor structure of UOV, we propose a variant in which multiple secret keys correspond to a single public key, and investigate its security and practicality.

Keywords: PQC, MPKC, UOV, advanced signatures, secret sharing

1. はじめに

現代の高度情報社会において, 情報セキュリティは重要な課題であり, 暗号はその根幹を担っている．しかし, 現在広く使われている RSA 暗号や楕円曲線暗号は実用的な規模の量子計算機を用いて解読できることが知られている．そこで, 量子計算機を用いても解読できない, かつ古典計算機上で動作する暗号として, 耐量子計算機暗号の構築が目下の課題となっている．

UOV (Unbalanced Oil and Vinegar) とその変種は, 有限体上の多変数多項式からなる連立方程式の求解困難性を用いた署名方式であり, 耐量子計算機暗号の有力な候補である．UOV とその変種は公開鍵暗号方式の署名であり, 鍵生成の段階で公開鍵と秘密鍵が生成される．公開鍵と秘密鍵の間には数学的な関係があり, 秘密鍵を有している主体のみが公開鍵による検証をパスできる署名を生成できる．

[1] 東京都立大学
Tokyo Metropolitan University
[2] 国立研究開発法人情報通信研究機構
National Institute of Information and Communications Technology
[a)] suzuki-toshihiro2@ed.tmu.ac.jp

UOV において, 公開鍵と秘密鍵の間の数学的な関係とは次のようなものである: 公開鍵が定義する空間 (アフィン多様体) 内に, 秘密鍵が定義する空間 (線形空間) が含まれる．そこでは次のような疑問が生じる: 公開鍵が定義する空間には, 最大でいくつの秘密空間を埋め込めるだろうか? 言い換えると, 1 つの公開鍵に最大でいくつの秘密鍵を対応させられるだろうか? 本研究はこの素朴な問いが出発点である．

本研究ではまず, 上記の問題に解答する．これにより, 一つの公開鍵に複数の秘密鍵が対応しているような一般化された UOV を与える (図 1)．また, この新しい方式の可能性を広げるため, 公開鍵に対応する秘密鍵を後から編集する動的メンバーシップを導入する．さらに, この方式の安全性と応用可能性について考察する．

本稿の貢献　本稿の主な貢献は以下の三点である．第一に, 一つの UOV 公開鍵に対応させられる秘密空間の個数を線形代数的に評価し, 複数の秘密鍵を生成する二種類の鍵生成アルゴリズムを構成する．第二に, 既存メンバーの秘密鍵を維持したままメンバーを追加・削除する動的メンバーシップを提案し, その効率と安全性を考察する．第三

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

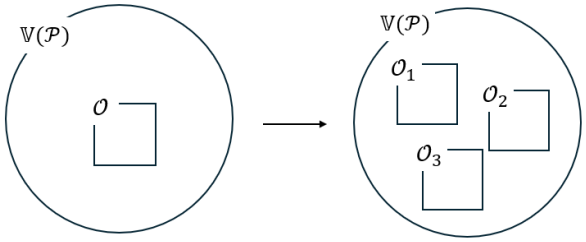


図 1: 公開鍵と秘密鍵のイメージ図. 左が従来の UOV で, 右が今回提案する変種. $\mathbb{V}(\mathcal{P})$ は公開鍵が定義する空間 (非線形) で, $\mathcal{O}$ は秘密鍵によって定義される空間 (線形).

に, 既知の秘密鍵から別の秘密鍵を復元する攻撃について考察するとともに, 組織署名や秘密分散への応用可能性を示す.

本稿の構成　本稿の構成は以下の通りである. 2 節で UOV に関する準備を行い, 3 節で UOV の鍵生成を一般化して MZ-UOV を定義する. 4 節で動的メンバーシップを提案し, 5 節でその安全性を考察する. 6 節で MZ-UOV の応用例を示し, 7 節で本稿をまとめる.

2. 準備

$\mathbb{F}_q$ を位数 q の有限体, $\boldsymbol{x}(n)$ を変数列 $(x_1, \ldots, x_n)^\top$ とし, しばしば単に $\boldsymbol{x}$ と書く. $\mathbb{F}_q[\boldsymbol{x}(n)]$ を $\mathbb{F}_q$ 上の n 変数多項式環, $\mathbb{F}_q[\boldsymbol{x}(n)]_2$ をその二次形式からなる部分集合, $\mathbb{F}_q[\boldsymbol{x}(n)]_2^m$ をその m 個の直積とする. $\mathbf{0}(n)$ は長さ n の零ベクトル, I, O はそれぞれ単位行列と零行列を意味する[*1].

UOV [1], [2] は署名方式であり, 鍵生成・署名・検証の 3 つのアルゴリズムから構成される. 鍵生成では以下を満たす鍵ペアを生成する.

定義 1 (UOV の鍵ペア)**.** $\mathcal{P} \in \mathbb{F}_q[\boldsymbol{x}(n)]_2^m$ と o 次線形空間 $\mathcal{O}$ に対し $\mathcal{P}(\mathcal{O}) = \mathbf{0}$ が成り立つとき, $\mathcal{P}, \mathcal{O}$ をそれぞれ $\mathrm{UOV}(q, n, m, o)$ の公開鍵, 秘密鍵という.

上の定義では $\mathcal{P}$ が代数側, $\mathcal{O}$ が幾何側にある. 説明を幾何側で揃えるなら, $\mathcal{P}(\mathcal{O}) = \mathbf{0}$ はアフィン多様体を用いて $\mathcal{O} \subset \mathbb{V}(\mathcal{P})$ と書ける. 鍵生成の具体的なアルゴリズムとしては以下のようなプログラムが効率的である [2].

プログラム 1: UOV.KeyGen.py

```
1  import sage.all as sg
2  def UOV_KeyGen(q, n, m, o, seed_sk, seed_pk):
3      K = UOVGF(q)
4      sg.set_random_seed(seed_sk)
5      O = sg.random_matrix(K, n-o, o)
6      sg.set_random_seed(seed_pk)
7      P1 = [Upper(sg.random_matrix(K, n-o, n-o))
           for _ in range(m)]
8      P2 = [sg.random_matrix(K, n-o, o) for _ in
           range(m)]
9      P3 = [Upper(-O.transpose()*P1[i]*O - O.
           transpose()*P2[i]) for i in range(m)]
10     P = [(P1[i].stack(sg.zero_matrix(K, o, n-o))
           ).augment(P2[i].stack(P3[i])) for i in
           range(m)]
11     return P, O
```

[*1] $O(*)$ で計算量を表すこともあり, 記号が衝突しているが, 本稿においてこれらを混同することは無いと思われる.

ここで `UOVGF(q)` は指定された位数 q の有限体を返し, `Upper(M)` は正方行列 M に対し $\boldsymbol{x}^\top M \boldsymbol{x} = \boldsymbol{x}^\top U \boldsymbol{x}$ となる上三角行列 U を返す.

署名ではダイジェスト $\mu \in \mathbb{F}_q^m$ に対し, 秘密鍵を用いて $\mathcal{P}(\sigma) = \mu$ を満たす $\sigma \in \mathbb{F}_q^n$ を求める. 検証では署名を公開鍵に代入して $\mathcal{P}(\sigma) = \mu$ の成立を確認する. 詳しくは [2] を見よ.

3. UOV 鍵生成の一般化

当初の問いに解答を与える. すなわち $\mathbb{V}(\mathcal{P})$ に埋め込める $\mathcal{O}$ の個数の最大値を求める. まず以下の数学の問題について考える.

問題 2 (放物線の決定)**.** xy 平面上の k 個の点 $p_1, \ldots, p_k$ が与えられる. このとき, $p_1, \ldots, p_k$ をすべて通る放物線 $y = ax^2 + bx + c$ を求めよ.

この問題を解くには, $f(x, y) := y - ax^2 - bx - c$ とし, 未知係数に関する連立方程式

$$\begin{cases} f(p_1) = 0, \\ \vdots \\ f(p_k) = 0 \end{cases}$$

を解けばいいのであった. 未知数が 3 個, 方程式が k 個なので, 一般に, $k < 3$ ならば条件を満たす放物線には自由度があり, $k = 3$ ならば放物線は一意に定まり, $k > 3$ ならば条件を満たす放物線は存在しない.

次に, 以下の問題について考える.

問題 3 (二次アフィン錐の決定)**.** $\mathbb{F}_q^n$ 上の k 個の o 次線形空間 $\mathcal{O}_1, \ldots, \mathcal{O}_k$ が与えられる. このとき, $\mathcal{O}_1, \ldots, \mathcal{O}_k$ をすべて含むようなアフィン多様体を定義する二次斉次多項式列 $\mathcal{P} \in \mathbb{F}_q[\boldsymbol{x}(n)]_2^m$ を求めよ.

$k = 1$ のとき, この問題は UOV の鍵生成そのものである. そのため, $k = 1$ の場合に関しては UOV の鍵生成アルゴリズムを適用すればよい.

一般の k では既存の UOV の鍵生成では上手くいかない. そこで, 放物線を決定したときと同様のアイデアを適用すれば問題を解決できる. $\mathcal{O}_l$ の基底を $\mathbf{b}_1^l, \ldots, \mathbf{b}_o^l$ とおき, 1 つの二次形式 $f \in \mathbb{F}_q[\boldsymbol{x}(n)]_2$ の未知係数に関する連立方程式

© 2026 Information Processing Society of Japan

$$\begin{cases} f(t_1\mathbf{b}_1^1+\cdots+t_o\mathbf{b}_o^1).\mathrm{coeff}(t_it_j)=0,\ (1\le i\le j\le o) \\ \vdots \\ f(t_1\mathbf{b}_1^k+\cdots+t_o\mathbf{b}_o^k).\mathrm{coeff}(t_it_j)=0\ (1\le i\le j\le o) \end{cases}$$

を解けばよい. ここで $f(t_1\mathbf{b}_1^1+\cdots+t_o\mathbf{b}_o^1).\mathrm{coeff}(t_it_j)$ は $f(t_1\mathbf{b}_1^1+\cdots+t_o\mathbf{b}_o^1)$ を $t_1,\ldots,t_o$ に関する多項式とみなした時の t_it_j の係数である.

未知係数は $\frac{n(n+1)}{2}$ 個, 多項式は $\frac{o(o+1)}{2}k$ 個である. 公開鍵としては, 係数列が線形独立な多項式が m 個必要 (すなわち方程式の解空間の次元が m 以上必要) であるため, $\frac{n(n+1)}{2}-\frac{o(o+1)}{2}k\ge m$ が要請される. (暗黙に方程式がフルランクであることを仮定している. そしてこれはほとんどの場合で成立する.) そのため, 条件を満たす k の最大値は

$$k_{\max}:=\left\lfloor\frac{n(n+1)-2m}{o(o+1)}\right\rfloor.$$

例えば $n=112, m=o=44$ であるならば, $k_{\max}=6$ である.

このような見方は $k=1$ の場合にも適用できる. UOV の鍵生成とは, 中心写像の変数変換であり, 巨大な線形部分を有する非線形なアフィン多様体の構築でもあり, 同時に, プロットされた点群を多項式で補間していると解釈することもできる. これは Shamir の秘密分散 [3] に似ているが, Shamir の秘密分散では補間多項式が秘匿されるのに対し, UOV では補間多項式が公開される.

これまでの議論に基づけば, 鍵生成において上記の連立方程式を解くことで, 一つの公開鍵に複数の秘密鍵が対応しているような一般化された UOV を構成できる. 署名と検証は従来のアルゴリズムと同じであり, これらの時間的な効率は変化しない. 公開鍵や秘密鍵一個あたりのデータサイズも変化しない. 大きなデメリットとして, 鍵生成アルゴリズムの時間的な効率が非常に悪い. 上記の連立方程式を解くには概ね

$$O\left(\left(\frac{o(o+1)}{2}k\right)^2\left(\frac{n(n+1)}{2}\right)\right)$$

の計算量を要する.

実装例として, 以下のようなものが考えられる.

プログラム 2: MZUOV.KeyGen1.py

```
import sage.all as sg
def MZUOV_KeyGen1(q,n,m,o,k,seed_sk, seed_pk):
    K = UOVGF(q)
    N = n*(n+1)//2
    sg.set_random_seed(seed_sk)
    sk = []
    for _ in range(k):
        O = sg.random_matrix(K, n, o)
        while O.rank() < o:
            O = sg.random_matrix(K, n, o)
        sk.append(O)
    M_blocks = [Constraint(O) for O in sk]
    M = Stack(M_blocks, K, N)
    J = KernelBasis(M, K, N)
    if J.ncols() < m:
        raise ValueError("dim ker(M) < m")
    sg.set_random_seed(seed_pk)
    while True:
        A = sg.random_matrix(K, J.ncols(), m)
        if A.rank() == m:
            break
    C = J*A
    pk = [QF(C.column(j), n) for j in range(m)]
    return pk, sk
```

表 1: 実行環境

項目	内容
ホスト OS	Windows 11
実行環境	WSL2（Ubuntu 22.04.2 LTS）
CPU	AMD Ryzen 7 5825U with Radeon Graphics（8 コア 16 スレッド）
メインメモリ	7.4 GiB（WSL2 から利用可能）
SageMath	Version 9.5
Python	Version 3.10.12
時間測定	`time.perf_counter()`
GPU	使用しない

ここで `Constraint(O)` は, $\mathcal{O}$ 上で二次形式が恒等的に零となるための $o(o+1)/2$ 個の線形条件を表す行列を返す. `Stack` はそれらの行列を縦に連結し, `KernelBasis(M)` は $\ker M$ の列基底行列を返す. また, `QF(c,n)` は長さ $n(n+1)/2$ の係数列を n 変数二次形式に変換する.

参考として, プログラム 2 を表 1 に示す環境で $(q,n,m,o,k)=(256,112,44,44,6)$ に対して実行すると, 行列生成からカーネル計算まで全て含めておよそ 68 から 69 秒程度を要する. ただし, 主要な線形代数処理であるカーネル計算が実際には 11 から 12 秒程度で完了しているため, 実装上の改善の余地があると考えられる.

特殊な場合として, $ko\le n$ の場合, 伝統的な UOV [1] の鍵生成に由来する次のような方法を用いることで, 従来の鍵生成と同等の計算量で鍵生成を行える. $n\approx 2.5o$ のパラメータを想定し, $k=2$ の場合について説明する.

(1)

$$\mathcal{S}=\begin{pmatrix} I_{n-2o} & S^{12} & S^{13} \\ O & I_o & S^{23} \\ O & S^{32} & I_o \end{pmatrix}$$

で表される正則行列 $\mathcal{S}$ をランダムに生成する.

(2)

© 2026 Information Processing Society of Japan

$$\mathcal{F}_i = \begin{pmatrix} F_i^{11} & F_i^{12} & F_i^{13} \\ O & O_{o\times o} & F_i^{23} \\ O & O & O_{o\times o} \end{pmatrix} (i = 1, \ldots, m)$$

で表現される中心写像 ([1]) $\mathcal{F}$ をランダムに生成する. ここで行列 F_i^{11} は $(n-2o)\times(n-2o)$ の上三角行列である.

(3) $\mathcal{P} = \mathcal{F} \circ \mathcal{S}$ を計算し, $\mathcal{P}$ を公開鍵,

$$\mathcal{S}^{-1}\begin{pmatrix} \mathbf{0}(n-2o) \\ \boldsymbol{x}(o) \\ \mathbf{0}(o) \end{pmatrix}, \mathcal{S}^{-1}\begin{pmatrix} \mathbf{0}(n-2o) \\ \mathbf{0}(o) \\ \boldsymbol{x}(o) \end{pmatrix},$$

を秘密鍵とする.

この方法のデメリットは, 一度に埋め込める秘密鍵の個数の上限が小さいことと, 秘密鍵をどのような値にするか事前に指定できないことである. 一方, 巨大な連立方程式を解く方法よりもはるかに高効率である.

実装例を以下に示す.

プログラム 3: MZUOV.KeyGen2.py

```
import sage.all as sg
def MZUOV_KeyGen2(q,n,m,o,seed_sk, seed_pk):
    K = UOVGF(q)
    v = n - 2*o
    Iv = sg.identity_matrix(K, v)
    Io = sg.identity_matrix(K, o)
    Zov = sg.zero_matrix(K, o, v)
    Zoo = sg.zero_matrix(K, o, o)
    sg.set_random_seed(seed_sk)
    while True:
        S12 = sg.random_matrix(K, v, o)
        S13 = sg.random_matrix(K, v, o)
        S23 = sg.random_matrix(K, o, o)
        S32 = sg.random_matrix(K, o, o)
        if not (Io - S32*S23).is_invertible():
            continue
        S1 = Iv.augment(S12).augment(S13)
        S2 = Zov.augment(Io).augment(S23)
        S3 = Zov.augment(S32).augment(Io)
        S = S1.stack(S2).stack(S3)
        break
    sg.set_random_seed(seed_pk)
    F = []
    coeffs = []
    while len(F) < m:
        F11 = Upper(sg.random_matrix(K, v, v))
        F12 = sg.random_matrix(K, v, o)
        F13 = sg.random_matrix(K, v, o)
        F23 = sg.random_matrix(K, o, o)
        F1 = F11.augment(F12).augment(F13)
        F2 = Zov.augment(Zoo).augment(F23)
        F3 = Zov.augment(Zoo).augment(Zoo)
        Fi = F1.stack(F2).stack(F3)
        ci = Coeff(Fi)
        C = sg.Matrix(K, coeffs + [ci])
        if C.rank() == len(coeffs) + 1:
            F.append(Fi)
            coeffs.append(ci)
    pk = [
        Upper(S.transpose()*Fi*S)
        for Fi in F
    ]
    E1 = sg.zero_matrix(K, v, o).stack(Io).stack(Zoo)
    E2 = sg.zero_matrix(K, v, o).stack(Zoo).stack(Io)
    Sinv = S.inverse()
    O1 = Sinv*E1
    O2 = Sinv*E2
    sk = [O1, O2]
    return pk, sk
```

ここで `Coeff(U)` は, 上三角行列 U の成分を

$$(U_{11}, U_{12}, \ldots, U_{1n}, U_{22}, \ldots, U_{nn})$$

の順に並べた係数列を返す関数である.

プログラム 3 を表 1 に示す環境で $(q,n,m,o,k) = (256, 112, 44, 44, 2)$ に対して実行すると, 約 3 秒で完了する. そのほとんどが中心写像の生成である.

これらの鍵生成により, 一つの UOV の公開鍵に複数の秘密鍵が対応しているような UOV の変種が構築できる. ここではこれを「MZ-UOV」[*2]と呼ぶことにする. 次のように定義できる.

定義 4 (MZ-UOV の鍵ペア)**.** $\mathcal{P} \in \mathbb{F}_q[\boldsymbol{x}(n)]_2^m$ と o 次線形空間 $\mathcal{O}_1, \ldots, \mathcal{O}_k$ $\left(k \le \left\lfloor \frac{n(n+1)-2m}{o(o+1)} \right\rfloor\right)$ に対し $\mathcal{P}(\mathcal{O}_i) = \mathbf{0}$ $(i = 1, \ldots, k)$ が成り立つとき, $\mathcal{P}, (\mathcal{O}_1, \ldots, \mathcal{O}_k)$ をそれぞれ MZUOV(q,n,m,o,k) の公開鍵, 秘密鍵という.

sk_1 と sk_2 が MZ-UOV の同一の公開鍵に対する秘密鍵であるとき, ここではこれらは互いに「姉妹鍵」[*3]であるということにする.

4. 動的メンバーシップ

UOV の一つの公開鍵を複数人で共有したいならば, 最も素朴なアイデアは, その秘密鍵をコピーして配布することである. このアイデアをここでは「コピー方式」と呼ぶことにする. コピー方式との差別化を図ることが, 今回のアイデアを有用なものにする上で重要となるだろう.

コピー方式に対する優位性を確保するためには, 鍵生成

[*2] Monozygotic-UOV の略. “monozygotic” は英語で「一卵性」を意味する. この名称は図 1 に由来する.

[*3] 英語では “sibling key” とすればよいが, 日本語では兄弟か姉妹のどちらかを選択する必要がある. そこで, 名詞に性がある言語を参考にすることにした. OpenAI GPT-5.6 sol を用いて, 話者数ランキング上位 20 位までの言語をリストアップし, その中の名詞に性を持つ言語によって鍵の性について多数決を取った結果, 僅差で鍵は女性ということになった.

© 2026 Information Processing Society of Japan

後にメンバーを後天的に追加・削除できたら便利である．そしてそれは以下のようにして実現できる．

4.1 メンバーの追加

公開鍵 pk に新しい秘密鍵 $\mathcal{O}_{\text{new}}$ を追加することを考える．pk を生成する際に `KeyGen1` で解くべき線形方程式の係数行列を

$$M = \begin{pmatrix} M_1 \\ \vdots \\ M_k \end{pmatrix}$$

とおく．ここで M_i $(i = 1, \ldots, k)$ はそれぞれ $\mathcal{O}_1, \ldots, \mathcal{O}_k$ が公開鍵に課す条件である．公開鍵の係数列は $\ker M$ から選択される．$\mathcal{O}_{\text{new}}$ が要請する条件 M_{new} を追加した新しい公開鍵 pk_{new} は

$$\ker M \cap \ker M_{\text{new}}$$

から選択すればよい．実際には, `KeyGen1` の際に計算する $\ker M$ の列基底を追加鍵 jk[*4]として保持しておいて,

$$\text{jk}_{\text{new}} = \text{jk} \cdot \ker\left(M_{\text{new}}\text{jk}\right)$$

を計算し, jk_{new} の張る空間から m 個の要素を取得すれば pk_{new} が得られる．

4.2 メンバーの削除

メンバー i の秘密空間 $\mathcal{O}_i$ のみを削除し, それ以外の秘密空間を維持することを考える．M から M_i を除いた係数行列を

$$M_{-i} = \begin{pmatrix} M_1 \\ \vdots \\ M_{i-1} \\ M_{i+1} \\ \vdots \\ M_k \end{pmatrix}$$

とおく．このとき, 係数列が

$$\mathbf{h}_i \in \ker M_{-i} \setminus \ker M_i$$

を満たす二次形式 h_i を削除鍵 rk_i[*5] として鍵生成時に保持しておく．(各 $i = 1, \ldots, k$ について, このような rk_i を事前に計算しておく.) 具体的には, $\ker M_{-i}$ から $M_i\mathbf{h}_i \neq \mathbf{0}$ となる $\mathbf{h}_i$ をランダムに探索する．h_i は以下を満たす:

$$\begin{cases} h_i(\mathcal{O}_j) = \mathbf{0} \ (j \neq i), \\ h_i(\mathcal{O}_i) \neq \mathbf{0}. \end{cases}$$

公開鍵を $\mathcal{P} = (p_1, \ldots, p_m)$ とし, 各 $j = 1, \ldots, m$ に対して $\alpha_j \in \mathbb{F}_q^*$ をランダムに選ぶ．更新後の公開鍵 $\mathcal{P}' = (p'_1, \ldots, p'_m)$ を

$$p'_j = p_j + \alpha_j h_i \qquad (j = 1, \ldots, m)$$

と定める．$\ell \neq i$ に対しては $M_\ell \mathbf{h}_i = \mathbf{0}$ であるため $\mathcal{P}'(\mathcal{O}_\ell) = \mathbf{0}$ が保たれる．一方, $M_i\mathbf{h}_i \neq \mathbf{0}$ かつ $\alpha_j \neq 0$ であるため, 各 p'_j は $\mathcal{O}_i$ 上で恒等的に零とはならない．したがって $\mathcal{P}'(\mathcal{O}_i) \neq \mathbf{0}$ となり, $\mathcal{O}_i$ は更新後の公開鍵に対する秘密鍵ではなくなる．

更新後は M_i と $\mathcal{O}_i$ をメンバーの一覧から除き, $\ker M_{-i}$ の列基底を新しい追加鍵 jk' とする．また, 残る各メンバー ℓ に対する削除鍵も

$$\mathbf{h}'_\ell \in \ker M_{-\{i,\ell\}} \setminus \ker M_\ell$$

から再計算する．これにより, 削除に関係しないメンバーは同じ秘密鍵を使い続けることができ, 以降のメンバー追加および削除にも対応できる．

4.3 評価

メンバーの追加・削除に必要なコストを評価する．

$$N := \frac{n(n+1)}{2}, \qquad r := \frac{o(o+1)}{2}, \qquad d := \dim \ker M$$

とおき, 追加後のカーネルの次元を d' とすると, メンバー追加の計算量は

$$O\left(\underbrace{rNd}_{M_{\text{new}}\text{jk の計算}} + \underbrace{rd\min\{r,d\}}_{\ker(M_{\text{new}}\text{jk}) \text{ の計算}} + \underbrace{Ndd'}_{\text{jk}_{\text{new}} \text{ の計算}} + \underbrace{mNd'}_{\text{pk}_{\text{new}} \text{ の生成}} \right).$$

である．ここで主要な処理は $M_{\text{new}}\text{jk}$ のカーネルと jk_{new} の計算である．一方, 事前計算した rk_i による公開鍵の切替は $O(mN)$ であり, メンバー削除自体は軽量である．

ただし, 更新後も追加と削除を可能にするには削除鍵を再計算する必要がある．更新後のメンバー数を K とすると, 現在の実装では全削除鍵の再計算に

$$O\big(K(K-1)^2 r^2 N\big)$$

を要し, これが主なボトルネックとなる．

鍵サイズを $\mathbb{F}_q$ の元の個数で数えると,

$$|\text{pk}| = mN, \quad |\text{sk}_i| = (n-o)o,$$
$$|\text{jk}| = Nd, \qquad |\text{rk}| = KN$$

となる．($|\text{sk}_i|$ は正規化表現を考慮している.) ビット数はこれらに $\lceil \log_2 q \rceil$ を掛けた値である．公開鍵と一人分の秘密鍵は通常の UOV と同じ大きさだが, 更新管理者は jk と全削除鍵を保持する必要がある．特に $d = \Theta(N)$ の場合, $|\text{jk}| = \Theta(N^2)$ となる．したがって, 本方式は各メンバーの保存量と署名・検証の効率を変えない一方, 更新管理者の前処理時間と保存量が大きい．

[*4] join key の略
[*5] revoke key の略.

© 2026 Information Processing Society of Japan

5. 安全性に関する考察

5.1 動的メンバーシップに対する攻撃

動的メンバーシップを有効活用するには, 必ず公開鍵の更新前後で変化しない秘密鍵が必要である. なぜなら, 公開鍵に対応する秘密鍵をすべて入れ替えるなら, 最初から鍵生成をやり直せばいいからである. このように, 変化しない秘密鍵が存在する鍵更新をここでは「ピボット」と呼び, ピボットによって変化しない秘密鍵を「ピボット軸」と呼ぶことにする.

ピボットを 1 回行うごとに, ピボット軸に対する鍵復元攻撃が容易になる. これは以下の二つの理由による.

MQ 問題の易化 ピボット軸 $\mathcal{O}_{\mathrm{axis}}$ に対してピボットを行うと, 新旧 2 つの公開鍵 $\mathcal{P}_{\mathrm{old}}, \mathcal{P}_{\mathrm{new}}$ について,

$$\begin{cases} \mathcal{P}_{\mathrm{old}}(\mathcal{O}_{\mathrm{axis}}) = \mathbf{0}, \\ \mathcal{P}_{\mathrm{new}}(\mathcal{O}_{\mathrm{axis}}) = \mathbf{0} \end{cases}$$

が成立することになる. これは $\mathcal{O}_{\mathrm{axis}}$ を復元するのに使用できる公開多項式が最大で 2 倍に増えることを意味する. これにより, UOV とその変種に対する既知の鍵復元攻撃である Reconciliation 攻撃 [4] においても, Intersection 攻撃 [5] においても, 後述の Rectangular MinRank 攻撃においても, その MQ 部分を構成する多項式が最大で 2 倍に増えることになる. 一般に, 変数の個数や次数が同じであれば, 使用できる多項式が多ければ多いほど MQ 問題の求解は容易になる. これは XL アルゴリズムや Gröbner 基底に基づく MQ 問題の求解において, より低い次数で求解次数に到達するようになるからである. なお, メンバー追加時には公開多項式が最大で m 増える一方, メンバー削除では新旧 $2m$ 個の公開多項式の線形包は高々

$$\mathrm{span}(p_1, \ldots, p_m, h_i)$$

である. ここで $p_1, \ldots, p_m$ は $\mathcal{P}_{\mathrm{old}}$ の要素, h_i は sk_i を削除するときにアルゴリズムの途中で計算する多項式である. したがってメンバー削除の場合はピボット軸に対する公開多項式は本質的に 1 個しか増えない.

Rectangular MinRank 攻撃の適用可能性 Rectangular MinRank 攻撃 [5], [6] はいくつかの UOV の変種を解読した実績のある鍵復元攻撃であり, いくつかの変種や一般化が知られている [7], [8], [9]. このうち [9] によって, 一般化 Rectangular MinRank 攻撃に対する強度に関する分類が導入されている.

$$s := \min\{i \in \mathbb{Z}_{\geq 1} \mid n - im < o\}.$$

としたとき, s が大きければ大きいほど, $\mathrm{UOV}(q, n, m, o)$ は一般化 Rectangular MinRank 攻撃に対して強い耐性を持つ. (適用は可能だが, より計算量が大きくなる条件でしか適用できなくなる.) i 回のメンバー追加ピボット後, 攻撃者はピボット軸に対して最大で $\mathrm{UOV}(q, n, (i+1)m, o)$ に相当する公開情報を得ることができるため, メンバー追加ピボット 1 回ごとに s が最大で 1 ずつ小さくなっていく. これは, より計算量が小さい条件で [9] の Rectangular MinRank 攻撃を適用できるようになることを意味する.

これらの理由により, 動的メンバーシップを安全に運用するには課題が多く, パラメータを慎重に選定した上でピボット回数に上限を設けた運用が必須となる. なお, メンバー削除はメンバー追加よりも本質的な公開多項式の増加が緩やかなので比較的安全である.

5.2 姉妹鍵復元攻撃

MZ-UOV に対する攻撃として, 既知の秘密鍵から同一の公開鍵に対応している別の秘密鍵を計算するというものが考えられる. つまり, sk_1 と sk_2 が pk に対応する秘密鍵であるとき, pk と sk_1 を知っている攻撃者が sk_2 の復元を試みる攻撃である. この攻撃がもたらす脅威は MZ-UOV の運用によって様々だが, 一つの秘密鍵から他の姉妹鍵を復元できるならば, MZ-UOV はコピー方式と同程度の機能しか提供できない.

秘密鍵が定義する線形空間は, 一般に, 互いに非自明な交差を持たない. そのため, 自身が持っている秘密空間内を探索して, 他の秘密空間内のベクトルを見つけ, One Vector 手法 [10] を適用するといった攻撃は適用できない. また, 自身が持っている秘密ベクトルを用いて One Vector 手法を適用すると, 自身の秘密空間しか復元できない.

ダイジェスト $\mu = \mathbf{0}$ に対して署名アルゴリズムを実行することで, $\mathbb{V}(\mathcal{P})$ 上の点を効率よく探索できるが, こうして得られたベクトルが必ずしも姉妹鍵の定義する秘密空間に含まれるわけではない. $\mathbb{V}(\mathcal{P})$ の次元は $n - m$ であり, その内の o 次線形空間に含まれるベクトルが得られる確率はおよそ $\frac{q^o}{q^{n-m}}$ であり, $(q, n, m, o) = (256, 112, 44, 44)$ では 2^{-192} となる.

これらの考察から, 我々の現時点での見解としては, この攻撃は困難であると考えている. しかし, 安全性証明等は得られておらず, 安全性について追加の議論を要する.

6. 応用に関する考察

本節では今回提案した MZ-UOV の応用可能性について考察する.

6.1 組織の署名, プロキシ署名, スペア鍵

まず, 最もシンプルな応用として, 集合的な人格を代表した署名というものが実現できる. ある集団が組織的な人格として MZ-UOV の公開鍵を一つ持っているとする. 組織内の同じ権限を持ったメンバーに秘密鍵を配布することで, これらのメンバーは組織を代表して署名できる. この

© 2026 Information Processing Society of Japan

とき, 具体的にどのメンバーが署名したかは匿名化される. メンバーの構成員が変化したとき, 動的メンバーシップによって公開鍵を更新することで, 追加/削除に関わらないメンバーには手元の秘密鍵を更新する必要性が生じない.

組織の署名以外にも, 仕組みを変えずに運用のみを変えることで, 他人への秘密鍵の貸与や, 秘密鍵のスペアを実現できる. 一度漏洩した秘密鍵は動的メンバーシップによって削除すればよい.

いずれの方法も, コピー方式との差別化を図るには公開鍵をピボットさせる必要がある. そのため, 前述のように時間・空間的なコストと安全性における課題が多い.

6.2 メンバーシップの証明

$\mathrm{sk}_A, \mathrm{sk}_B$ を pk に対応する秘密鍵とする. Alice が sk_A を, Bob が sk_B を所有しているとする. このとき, Alice と Bob は以下のようにして互いが同じ秘密鍵グループに属していることを証明できる.

(1) Alice がチャレンジ μ_A を生成し, Bob に渡す. Bob は sk_B を用いて, 署名アルゴリズムにより $\mathcal{P}(\sigma_A) = \mu_A$ となるような σ_A を計算し, Alice に渡す.
(2) Alice は受け取った σ_A を検証することで, Bob が pk に対応する秘密鍵を持っていることを確認する.
(3) 同様に Bob からもチャレンジ μ_B を Alice に渡し署名させることで, Alice のメンバーシップを確認する.

このようにして互いのメンバーシップを証明し合うことができる. しかし, これ自体はコピー方式でも実現できる. そこで次のような状況を考える.

6.3 秘密分散の安全な復元

ある情報 s を Alice と Bob の同意があるときのみ二人に開示したい. そこで $\mathrm{s} = \mathrm{sk}_A \oplus \mathrm{sk}_B$ によって秘密分散を行い, sk_A を Alice に, sk_B を Bob に渡す. ここで悪意の第三者 Eve による次のような中間者攻撃が考えられる: Eve は Bob になりすまして Alice から sk_A を受け取り, さらに Alice になりすまして Bob から sk_B を受け取ることで, s を復元する. この攻撃に対応するためには次のようにすればよい.

(1) ディーラーは, 秘密分散 $\mathrm{s} = \mathrm{sk}_A \oplus \mathrm{sk}_B$ を計算したのち, sk_A と sk_B の両方に対応する公開鍵 pk を計算し, 公開する. sk_A を Alice に, sk_B を Bob に渡す.
(2) Alice と Bob は秘密を復元する前に, 前述のメンバーシップ証明プロトコルによって, 互いが秘密分散の正規の参加者であることを確認する.
(3) 互いのメンバーシップが確認されたときのみシェアを開示し, 秘密を復元する.

この方法によって, なりすましが懸念される状況下でも安全に秘密分散の復元が可能となる. また, 開示された sk_A や sk_B を再度公開鍵に適合するか検証することで, (姉妹鍵のいずれでもないような) 偽のシェアでないことを確認できる. 本人確認を行うには通常の署名でも可能だが, 今回の方法では互いを匿名化したまま, 秘密分散の正規の参加者であることのみを証明できる. このプロトコルは配布される秘密鍵がすべて同じ値であるコピー方式では実現できないため, MZ-UOV によるコピー方式に対する明確な差別化である. 今回は簡単のためシェアの総数を 2 で説明したが, 理論上 $k_{\max}$ までシェアを増やすことができる. また, この方法は秘密分散のアルゴリズムには依存せず, 閾値秘密分散 [3] にも適用できる. 注意として, この方法はディーラーの悪意を検出しないので, 検証可能秘密分散 [11] とは異なる安全性を提供している. この方式をここでは「証明書付き秘密分散」と呼ぶことにする.

7. まとめ

本稿ではまず, UOV の一つの公開鍵に対応させることのできる異なる秘密鍵の個数について, その最大値を求めた. その結果, その値は 1 より大きく, UOV の公開鍵には同時に複数の秘密鍵を対応させることが可能であることが明らかになった. その時に利用したアイデアをそのまま利用することで, 一般化された鍵生成 KeyGen1 を構築した. また, KeyGen1 が低速であることを考慮して, より速いがいくつかの制限を持つ別の鍵生成 KeyGen2 を構築した. これらによって新しい UOV の変種 MZ-UOV を提案した. また, MZ-UOV の可能性を広げるために動的メンバーシップを構築した.

次に MZ-UOV の安全性について考察した. まず, 動的メンバーシップには安全性上の課題があることを指摘した. また, MZ-UOV に固有の攻撃として姉妹鍵復元攻撃があり, 現時点で有効な攻撃アルゴリズムは見つかっていないが, 今後さらなる安全性評価が必要であることを指摘した.

最後に MZ-UOV の有用性について考察した. 組織の署名, プロキシ署名, スペア鍵は動的メンバーシップを使用するため課題が多いが, 証明書付き秘密分散は安全性に課題のある動的メンバーシップを使用せず, さらにコピー方式では実現できない方式であるため, MZ-UOV に最も適した応用の一つと考えられる.

今後の課題と展望は以下の通りである.

証明書付き秘密分散の安全性評価 証明書付き秘密分散の実用化に伴う課題は, まず姉妹鍵復元攻撃に対する安全性を評価することである. もし姉妹鍵を容易に復元できてしまうなら, 秘密分散自体の安全性も脅かすことになるため, 慎重に評価する必要がある.

KeyGen1 を用いる場合は鍵生成が遅く, KeyGen2 を用いる場合はシェアの総数が少ない上に, 復元される秘密を鍵生成前に指定できない. これらのトレードオフのうち, どちらのデメリットを受け入れるかは運用次第である.

また, 現在提案されている UOV のパラメータは近年強

© 2026 Information Processing Society of Japan

力な攻撃 [12] が提案されたため流用すべきではなく, 追加のパラメータ選定が必要である.

MAYO 等への適用 MAYO [13] や QR-UOV [14] は UOV の公開鍵のデータサイズを削減するために導入された変種である.

公開鍵のデータとは二次形式の係数列のことである. そのため, これらの変種に今回のアイデアを適用することで, 鍵生成の際の未知係数を減らし, ボトルネックであった KeyGen1 の効率を改善する可能性がある. さらに, MAYO のように n に対して o が小さい場合, $k_{\max}$ も飛躍的に増加する. なぜなら $k_{\max}$ は概ね $\left(\frac{n}{o}\right)^2$ であるからである. このように, UOV の変種と今回のアイデアは相性がよく, MZ-MAYO 等の開発を進めていく価値は大きいと考えられる.

謝辞 本研究は, JST 経済安全保障重要技術育成プログラム (JPMJKP24U2), JSPS 科研費 JP24K14949, JP25K21204 の助成を受けたものである.

参考文献

[1] Kipnis, A., Patarin, J. and Goubin, L.: Unbalanced oil and vinegar signature schemes, *International Conference on the Theory and Applications of Cryptographic Techniques*, Springer, pp. 206–222 (1999).

[2] Beullens, W., Chen, M.-S., Ding, J., Gong, B., Kannwischer, M. J., Patarin, J., Peng, B.-Y., Schmidt, D., Shih, C.-J., Tao, C. et al.: UOV: Unbalanced Oil and Vinegar, *Specification Document of NIST PQC Standardization Additional Digital Signature Schemes Round 2* (2025).

[3] Shamir, A.: How to share a secret (1979).

[4] Ding, J., Yang, B.-Y., Chen, C.-H. O., Chen, M.-S. and Cheng, C.-M.: New Differential-Algebraic Attacks and Reparametrization of Rainbow, *Applied Cryptography and Network Security – ACNS 2008*, Springer, pp. 242–257 (2008).

[5] Beullens, W.: Improved cryptanalysis of UOV and Rainbow, *Advances in Cryptology – EUROCRYPT 2021*, Springer, pp. 348–373 (2021).

[6] Furue, H. and Ikematsu, Y.: A new security analysis against MAYO and QR-UOV using rectangular MinRank attack, *Advances in Information and Computer Security – IWSEC 2023*, Springer, pp. 101–116 (2023).

[7] Guo, H., Jin, Y., Pan, Y., He, X., Gong, B. and Ding, J.: Practical and Theoretical Cryptanalysis of VOX, *Post-Quantum Cryptography – PQCrypto 2024*, Springer, pp. 186–208 (2024).

[8] Suzuki, T., Furue, H., Ito, T., Nakamura, S. and Uchiyama, S.: An Extended Rectangular MinRank Attack against UOV and Its Variants, *Advances in Information and Computer Security – IWSEC 2025*, Springer, pp. 111–130 (2025).

[9] Suzuki, T., Ito, T. and Uchiyama, S.: Rectangular MinRank 攻撃の幾何的な考察に基づく一般化とその UOV と MAYO への適用, *2026 Symposium on Cryptography and Information Security – SCIS2026* (2026).

[10] Pébereau, P.: One vector to rule them all: Key recovery from one vector in UOV schemes, *Post-Quantum Cryptography – PQCrypto 2024*, Springer, pp. 92–108 (2024).

[11] Chor, B., Goldwasser, S., Micali, S. and Awerbuch, B.: Verifiable secret sharing and achieving simultaneity in the presence of faults, *26th Annual Symposium on Foundations of Computer Science (sfcs 1985)*, IEEE, pp. 383–395 (1985).

[12] Furue, H. and Ikematsu, Y.: Key Recovery Attacks on UOV Using p^l-truncated Polynomial Rings, *Cryptology ePrint Archive* (2026).

[13] Beullens, W., Campos, F., Celi, S., Hess, B. and Kannwischer, M. J.: MAYO, *Specification Document of NIST PQC Standardization Additional Digital Signature Schemes Round 2* (2025).

[14] Furue, H., Ikematsu, Y., Hoshino, F., Takagi, T., Yasuda, K., Miyazawa, T., Saito, T. and Nagai, A.: QR-UOV, *Specification Document of NIST PQC Standardization Additional Digital Signature Schemes Round 2* (2025).

© 2026 Information Processing Society of Japan